```
# render_svg.py
# Renders a DXF to SVG.
#
# Typical pipeline:
# 1. python render_svg.py input.dxf → drawing.svg
# 2. python rasterise_tiles.py --svg drawing.svg → tiles/ + tile_meta.json
# 3. python extract_manifest.py --dxf input.dxf \ → hitboxes.json
#    --labels labels.txt \
#    --tile-meta tile_meta.json
#
# Usage:
#   python render_svg.py input.dxf [output.svg] [--text-to-path]
#
#   --text-to-path  Convert all DXF text/MTEXT to filled outline paths in the
#                   SVG rather than <text> elements. Use this when your DXF
#                   fonts are not available on the viewing machine, or when you
#                   need pixel-accurate glyph rendering. Note: extract_manifest
#                   cannot read paths as text, so it will fall back to DXF
#                   coordinate matching (unaffected by this flag).
#
# Outputs:
#   output.svg  -- vector SVG via ezdxf SVGBackend

import argparse
import sys

import ezdxf
from ezdxf.addons.drawing import Frontend, RenderContext
from ezdxf.addons.drawing.layout import Margins, Page, Settings, Units
from ezdxf.addons.drawing.svg import SVGBackend
from ezdxf.bbox import extents as bbox_extents

# ---- CLI -------------------------------------------------------------------

parser = argparse.ArgumentParser(
    description="Render DXF to SVG and write transform.json."
)
parser.add_argument("dxf", nargs="?", default="test_diagram.dxf",
                    help="Input DXF file (default: test_diagram.dxf)")
parser.add_argument("svg", nargs="?", default=None,
                    help="Output SVG file (default: <dxf_stem>.svg)")
parser.add_argument("--text-to-path", action="store_true",
                    help="Convert text/MTEXT to outline paths instead of "
                         "<text> elements (font-independent, path-accurate)")
```

```
args = parser.parse_args()

dxf_path  = args.dxf
svg_path  = args.svg or (dxf_path.rsplit(".", 1)[0] + ".svg")
text_to_path = args.text_to_path

print(f"DXF          : {dxf_path}")
print(f"SVG          : {svg_path}")
print(f"text-to-path : {text_to_path}")

doc = ezdxf.readfile(dxf_path)
msp = doc.modelspace()

# ---- 1. DXF extents via entity bbox scan ------------------------------------
# Never trust $EXTMIN/$EXTMAX -- often sentinel values (~1e20)

print("Scanning entity extents...")
bbox = bbox_extents(msp)
if bbox is None or not bbox.has_data:
    sys.exit("ERROR: Could not determine drawing extents")

dxf_x_min, dxf_y_min = bbox.extmin.x, bbox.extmin.y
dxf_x_max, dxf_y_max = bbox.extmax.x, bbox.extmax.y
dxf_w = dxf_x_max - dxf_x_min
dxf_h = dxf_y_max - dxf_y_min

print(f"DXF extents  : x=[{dxf_x_min:.4f}, {dxf_x_max:.4f}] "
      f"y=[{dxf_y_min:.4f}, {dxf_y_max:.4f}]")
print(f"DXF size     : {dxf_w:.4f} x {dxf_h:.4f} units")

# ---- 2. Build render Settings -----------------------------------------------
# text_policy / text_as_paths controls whether text is emitted as <text>
# elements or converted to filled <path> glyphs in the SVG output.
# We try the modern API first (ezdxf >= 1.1) and fall back gracefully.

def _make_settings(text_to_path: bool) -> Settings:
    if not text_to_path:
        return Settings()

    # Modern API (ezdxf >= 1.1): Settings(text_policy=TextPolicy.FILLING)
    try:
        from ezdxf.addons.drawing.properties import TextPolicy
        print("text-to-path : using TextPolicy.FILLING")
        return Settings(text_policy=TextPolicy.FILLING)
    except (ImportError, AttributeError, TypeError):
        pass
```

```
    # Older API: Settings(text_as_paths=True)
    try:
        s = Settings(text_as_paths=True)
        print("text-to-path : using text_as_paths=True")
        return s
    except TypeError:
        pass

    # Last resort: show_text attribute
    s = Settings()
    if hasattr(s, "show_text"):
        s.show_text = False
        print("text-to-path : using show_text=False (fallback)")
    else:
        print("WARNING: text-to-path not supported by this ezdxf version -- "
              "text will remain as <text> elements.", file=sys.stderr)
    return s

settings = _make_settings(text_to_path)

# ---- 3. Light mode colour overrides ----------------------------------------
# ezdxf's default render context uses a dark background with light-coloured
# entities. For light mode we need:
#   • Background → white (#ffffff)
#   • ACI colour 7 ("white" on dark themes, renders as pink) → black (#000000)
#   • All other entity colours unchanged.
#
# RenderContext exposes set_colors() on the current layout properties to
# override the background, and allows a custom CSS rewrite via the SVGBackend
# defs after rendering. The most reliable cross-version approach is to patch
# the resolved CSS classes in the SVG string directly after get_string().

def _apply_light_mode(svg_string: str) -> str:
    """
    Post-process the SVG string to enforce light mode:
      1. Replace the dark background rectangle fill with white.
      2. Replace the ezdxf CSS class for ACI-7 (near-white / pink on dark)
         with black so it reads as dark text on a white background.
    """
    import re

    # ezdxf writes ACI colour classes as .C0, .C1, … .C255 in a <style> block.
    # ACI 7 = white-on-dark. On a white background it should be black.
    # ACI 0 = black (already correct for light mode — leave untouched).
    #
    # Strategy A: rewrite the CSS fill/stroke for .C7 to #000000.
    # Strategy B: also force the SVG background rect to white.
```

```
    # -- Background colour --
    # ezdxf emits a <rect ... fill="#rrggbb"/> as the first rect (background).
    # Replace any very-dark background fills with white.
    # Matches: fill="#000000", fill="#0d0d0d", fill="#1a1a1a" etc.
    svg_string = re.sub(
        r'(<rect\b[^>]*\bfill=")#(?:[01][0-9a-fA-F]{5})("[^>]*>)',
        r'\g<1>#ffffff\g<2>',
        svg_string,
        count=1,          # only the background rect (first occurrence)
    )

    # -- ACI-7 (white / near-white) → black in the <style> block --
    # ezdxf emits something like:
    #   .C7{fill:#ffffff;stroke:#ffffff}   (white on dark)
    # We rewrite that class to black so text/lines are visible on white.
    svg_string = re.sub(
        r'(\.C7\s*\{[^}]*?)#(?:f{6}|f{5}e|fffffe|fefefe)([^}]*\})',
        r'\g<1>#000000\g<2>',
        svg_string,
        flags=re.IGNORECASE,
    )

    # Also handle stroke separately in case ezdxf splits fill and stroke
    # across multiple .C7 rules.
    svg_string = re.sub(
        r'(\.C7\b[^{]*\{(?:[^}]*?(?:fill|stroke):[^;#}]*)*(?:fill|stroke):)#(?:f{
        r'\g<1>#000000',
        svg_string,
        flags=re.IGNORECASE,
    )

    return svg_string

# ---- 4. Render SVG ------------------------------------------------------

print("Rendering SVG...")

ctx = RenderContext(doc)

# Set white background via layout properties (ezdxf >= 0.17).
# This affects the backend's background rect colour.
try:
    ctx.current_layout_properties.set_colors(bg="#ffffff")
    print("Background   : set to #ffffff via set_colors()")
except AttributeError:
    print("Background   : set_colors() unavailable -- will patch via post-process
```

```
backend = SVGBackend()
Frontend(ctx, backend).draw_layout(msp)

page = Page(0, 0, Units.mm, Margins(0, 0, 0, 0))
svg_string = backend.get_string(page, settings=settings)

# Apply light mode post-processing (catches anything set_colors didn't cover).
svg_string = _apply_light_mode(svg_string)

with open(svg_path, "w", encoding="utf-8") as f:
    f.write(svg_string)

print(f"SVG written  : {svg_path}")
print()
print("── Next step ──────────────────────────────────────────────────────────")
print(f"  python rasterise_tiles.py --svg {svg_path}")
print()
print("  rasterise_tiles.py will:")
print("  • rasterise the SVG to a high-res PNG tile pyramid (tiles/)")
print("  • write tile_meta.json for DXFViewer")
```